Streamlit Features Explained

Streamlit is a **Python framework for building interactive web applications**, especially dashboards, analytics tools, data viewers, and AI applications.

Since you're already using **Python + pandas + SQL + Streamlit** for your SME Intelligence Hub / Financing Leads work, you can think of it like this:

> **Python does the processing. Streamlit turns that Python into a web application.**

For example, instead of only doing:

Code — Python

```
df.head()
```

you can display it on a web page:

Code — Python

```
st.dataframe(df)
```

What can Streamlit do?

Here are the main features that are useful for the kind of internal applications you're building.

	Feature	Streamlit	Example
1	Text	`st.title()`, `st.header()`, `st.markdown()`	Dashboard headings
2	Tables	`st.dataframe()`	Customer / financing data
3	Metrics	`st.metric()`	Total customers, exposure
4	Charts	`st.bar_chart()`, `st.line_chart()`	Portfolio analysis
5	Buttons	`st.button()`	Run analysis
6	Dropdown	`st.selectbox()`	Select customer
7	Multi-select	`st.multiselect()`	Select sectors
8	Search/Text	`st.text_input()`	Search company
9	Number input	`st.number_input()`	Financing amount
10	Date	`st.date_input()`	Reporting date

11	Upload	`st.file_uploader()`	Upload Excel/CSV
12	Download	`st.download_button()`	Download results
13	Tabs	`st.tabs()`	Separate dashboard sections
14	Sidebar	`st.sidebar`	Navigation/filtering
15	Columns	`st.columns()`	Dashboard layout
16	Forms	`st.form()`	User input form
17	Dialog	`st.dialog()`	Pop-up style interactions
18	Progress	`st.progress()`	Processing indicator
19	Status	`st.success()`, `st.error()`	User feedback
20	Multipage apps	`pages/` or navigation APIs	MIS / UDX / Financing pages
21	Session state	`st.session_state`	Remember selections
22	Caching	`st.cache_data`	Improve performance

Example: Your Financing Leads app

You could eventually build something like:

Code — Plain Text

```
               BANK LOGO
            Financing Leads

------------------------------------------------

Total Companies    Total Leads     Total Financing
    1,250              186             RM450m

------------------------------------------------

State:      [Selangor ▼]
Sector:     [Manufacturing ▼]
Rating:     [All ▼]

------------------------------------------------

Company           Sector          Score
ABC Sdn Bhd       Manufacturing    87
XYZ Sdn Bhd       Technology       82

------------------------------------------------

          Financing Leads by Sector
                  [Chart]
```

All of that can be built with Streamlit.

---

1. Display your DataFrame

Instead of:

Code Python

```
df.head()

```

use:

Code Python

```
st.dataframe(df.head())

```

Or:

Code Python

```
st.dataframe(
    df,
    use_container_width=True,
    hide_index=True
)

```

This gives users an interactive table.

---

2. Create KPI cards

Very useful for your dashboards:

Code Python

```
col1, col2, col3 = st.columns(3)

col1.metric(
    label="Total Companies",
    value="1,250"
)

col2.metric(
    label="Potential Leads",
    value="186"
)

col3.metric(
    label="Financing Potential",
    value="RM450M"
)

```

---

3. Create dropdown filters

For example:

Code Python

```python
sector = st.selectbox(
    "Select Sector",
    df["Sector"].unique()
)

filtered_df = df[df["Sector"] == sector]

st.dataframe(filtered_df)

```

You could have:

Code Plain Text

```
Select Sector
[ Manufacturing ▼ ]

Company
ABC Manufacturing
DEF Industries
XYZ Engineering

```

---

4. Search function

This would be very useful for SME Intelligence Hub.

Code Python

```python
search = st.text_input("Search Company")

if search:

    result = df[
        df["Company Name"]
        .str.contains(search, case=False, na=False)
    ]

    st.dataframe(result)

```

Users could type:

Code Plain Text

```
PETRONAS

```

and get matching companies.

---

5. Upload Excel / CSV

Streamlit can let users upload data directly.

Code Python

```
uploaded_file = st.file_uploader(
    "Upload File",
    type=["xlsx", "csv"]
)

if uploaded_file:

    df = pd.read_excel(uploaded_file)

    st.dataframe(df)

```

This is useful if your users need to upload **customer lists, survey files, UDX files, or assessment data**.

---

6. Download results

You can also allow users to download filtered data:

Code Python

```
csv = filtered_df.to_csv(index=False)

st.download_button(
    "Download Results",
    csv,
    "financing_leads.csv",
    "text/csv"
)

```

---

7. Tabs

You can make your application much cleaner:

Code Python

```
tab1, tab2, tab3 = st.tabs([
    "Overview",
    "Financing Leads",
    "Customer Details"
])

with tab1:
    st.write("Dashboard")

with tab2:
    st.dataframe(df)

with tab3:
    st.write("Customer Information")

```

---

8. Sidebar

You were already working with this recently.

Example:

Code Python

```python
st.sidebar.title("Navigation")

sector = st.sidebar.selectbox(
    "Sector",
    df["Sector"].unique()
)

state = st.sidebar.selectbox(
    "State",
    df["State"].unique()
)

```

A good layout for your application would be:

Code Plain Text

```
Sidebar                 Main Area

Main Dashboard          Logo
MIS                     SME Intelligence Hub
UDX
Financing Leads         KPI KPI KPI KPI
CCPT
AI Survey               Charts
                        Data Table

```

---

9. Multipage Streamlit application

You can separate your system into multiple Python files:

Code Plain Text

```
SME_Intelligence_Hub/

main.py

pages/
    1_MIS.py
    2_UDX.py
    3_Financing_Leads.py
    4_CCPT.py
    5_AI_Survey.py

```

Then run:

Code BAT

```
streamlit run main.py

```

Streamlit handles navigation between the pages.

---

10. Connect Streamlit to SQL Server

This is probably one of the **most useful features for you**.

The architecture could become:

Code Plain Text

```
SQL Server
    ↓
Python
    ↓
pandas DataFrame
    ↓
Streamlit
    ↓
Internal Web Application

```

For example:

Code Python

```
import streamlit as st
import pandas as pd
import pyodbc

conn = pyodbc.connect(
    "DRIVER={ODBC Driver 17 for SQL Server};"
    "SERVER=YOUR_SERVER;"
    "DATABASE=YOUR_DATABASE;"
    "Trusted_Connection=yes;"
)

query = """
SELECT *
FROM customer
"""

df = pd.read_sql(query, conn)

st.title("Customer Dashboard")

st.dataframe(df)

```

Now instead of manually extracting SQL data, the app can display it directly.

---

11. Charts

Streamlit supports quick charts.

Code Python

```
st.bar_chart(df.groupby("Sector")["Exposure"].sum())

```

Or:

Code Python

```
st.line_chart(df["Outstanding"])

```

For more sophisticated visualisation, you can combine Streamlit with libraries such as **Plotly, Altair, and Matplotlib**.

For example:

Code Python

```python
import plotly.express as px

fig = px.bar(
    df,
    x="Sector",
    y="Exposure"
)

st.plotly_chart(
    fig,
    use_container_width=True
)

```

---

12. Session State

This becomes important once your app becomes larger.

Code Python

```python
if "username" not in st.session_state:
    st.session_state.username = ""

st.session_state.username = "Syahid"

```

Streamlit can then remember values while the user moves around the application.

This is useful for:

- selected customer
- filters
- logged-in user
- selected reporting date
- previous calculation
- form values

---

13. Caching

If your SQL query takes 10 seconds every time, you don't necessarily want Streamlit to rerun it constantly.

You can cache the result:

Code Python

This can make dashboards significantly faster.

```python
@st.cache_data
def load_data():

    df = pd.read_sql(query, conn)

    return df
```

---

14. Forms

For example, a financing lead assessment:

Code — Python

```python
with st.form("lead_form"):

    company = st.text_input("Company Name")

    sector = st.selectbox(
        "Sector",
        ["Manufacturing", "Services", "Technology"]
    )

    financing = st.number_input(
        "Financing Required"
    )

    submitted = st.form_submit_button(
        "Submit"
    )

if submitted:
    st.success("Lead submitted successfully")

```

---

15. Streamlit can also become an AI application

This is where Streamlit gets especially interesting for your **SME Intelligence Hub**.

It has chat-style components such as:

Code — Python

```python
st.chat_input()

```

and:

Code — Python

```python
st.chat_message()

```

So conceptually your system could become:

Code — Plain Text

```
                SME Intelligence Hub
                         |
          ┌──────────────┼──────────────┐
          ↓              ↓              ↓
        MIS            UDX        Financing Leads
          |              |              |
          └──────────────┼──────────────┘
                         ↓
```

```
                  AI Assistant
                       ↓
                SQL / Documents

```

A user could ask:

> "Show me manufacturing companies with high financing potential in Selangor."

Your Python backend could query/filter the data and Streamlit could present the result.

For your current project, I'd focus on these next

Since you've already got the basic Streamlit pages working, I'd learn these roughly in this order:

1. Layout

Code Python

```python
st.columns()
st.container()
st.tabs()

```

2. Data display

Code Python

```python
st.dataframe()
st.metric()

```

3. Filtering

Code Python

```python
st.selectbox()
st.multiselect()
st.text_input()

```

4. Navigation

Code Python

```python
st.navigation()
st.Page()

```

5. SQL integration

6. Charts / Plotly

7. `st.session_state`

8. `@st.cache_data`

9. Authentication / authorization

10. AI chatbot integration

For the **SME Intelligence Hub**, this combination can take you quite far: **SQL Server → pandas → Streamlit → Plotly → internal**

web app, with AI added later if needed.